Oracle vs PostgreSQL - mab

	Oracle	**PostgreSQL**	비고
출생년도	1977년 SDL 스타트업	1982년 UC Berkeley **Ingres** project	1970년 Codd가 RDB 이론 발표 직후에 Ingres 프로젝트 시작
출생지	Menlo Park, California (**SDL**) Oracle Corporation 2300 Oracle Way Austin, TX 78741, USA	University of California, Berkeley Berkeley, CA 94720, USA	
창시자	Larry Ellison	Michael Stonebraker	
출시년도	1979년 Oracle DB v2 출시	1996년 PostgreSQL fos 출시	
목적	산업용	교육연구용	1994년 Postgres95 프리오픈소스
배경	1970년대 RDB 붐 기술 시대 흐름 속에서 등장한 독립 스타트업	PostgreSQL은 Michael Stonebraker가 UC Berkeley에서 만든 Postgres 연구 프로젝트가 기반	버클리 (Ingres 연구) ↘ 같은 시대 기술 흐름 (RDB 붐) ↙ SDL (상용 DB 시장 먼저 잡음)
SQL도입	IBM System R 영향 SQL중심 설계로 산업포준화됨 표준 언어를 잡은 쪽이 생태계를 장악	QUEL이라는 자체 언어 기반 초기엔 SQL 약했으나 최근에는 ISO SQL 표준에 가장 충실하게 준수함	SQL기원은 IBM 연구소의 System R 프로젝트 SEQUEL Donald D. Chamberlin Raymond F. Boyce
SQL표준	이미 고객이 Oracle SQL에 익숙해짐, “우리가 표준이다”	표준 SQL 중심 설계 기능 추가 시 ISO SQL 기준 먼저 확인	1980년 비표준 혼돈의 시대 1986~1987: 공식 표준 등장
SQL표준준수	약 145~155개 수준 준수율 : 80~88%	약 170개 이상 준수 준수율 : 95% 이상	2023년 ISO SQL표준 필수 항목 177개 기준 준수율
동시성	Undo/Redo + ITL 구조로 hotspot row update 경쟁 상황에서 더 안정적	읽기/쓰기 충돌이 적고 read-heavy OLTP에 강함	오라클은 undo/redo, PG는 mvcc 스냅샷 기반으로 ACID 동시성 만족함 *ACID : Atomic, Consistency, Isolation, Duration*
튜닝	힌트(HINT), 인덱스 설계, 세밀한 옵티마이저 제어 중심	실행계획/통계 기반 튜닝 + SQL 재작성 비중 큼	오라클이 SQL튜닝 면에서 상대적으로 강력함
튜닝툴	XPLAN, trace, explain, tkprof 등등 다양한 튜닝방법 존재	explain, analyze 외에 없음	

Hot block	buffer busy waits와 gc buffer 3-way busy로 인한 활성세션 폭주(cache fusion) RAC노드 전체 대기상태 발생	네트워크 cache fusion 비용 없음 global block ownership 없음 gc 3-way 없으나 row lock wait 발생으로 writer 대기상태 발생	"모든 세션이 특정 블록에 집중 접근" SNS 조회수 업데이트문 폭주 발생시 오라클 보다 PG가 유리하지만, 최고의 구조는 redis - kafka - batch - async
ITL	블록 여유공간이 허용하는 만큼 허용TX수 조정 가능(8K) INITRANS : 처음부터 확보하는 ITL 개수 MAXTRANS : 과거엔 최대 개수 제한(현재는 사실상 무시)	설정 불가능	"같은 블록에서 동시에 수정 가능한 트랜잭션 슬롯 수" 오히려 실무에서 ITL 무한 증가시 단점 다수 발생. 블록 헤더 공간 증가 *row* 저장 공간 감소 *block split* 증가 가능 테이블/인덱스 크기 증가
LATERAL	12c버전 부터 lateral join 지원하지만 성능 약함	Nested Loop + Index Scan + limit 패턴의 lateral 조인 효율 강함	Infinity scroll UX (SNS)의 TOP-N 대량 읽기에 PostgreSQL 성능 우위
서브쿼리	scalar subquery unnesting이 과 caching 으로 성능 좋음	subquery caching 지원 안함 lateral join이 더 유리함	
힌트	수 많은 힌트 지원	EDB, EPAS 에서 일부 힌트 지원	오라클 PARALLEL힌트에 비해 PG는 동작 하지 않은 경우가 많음
기본권한	기능없음. 테이블 생성하면 GRANT 해야하는 번거로움	Default privilege설정으로 테이블 생성시 별도 GRANT 필요 없음	
Tablespace	테이블,인덱스,LOB 및 업무별로 생성하는 오랜 문화, 테이블스페이스 설계가 필수	pg_default 만 사용해도 성능이나 유지관리에 아무 지장이 없음	PG는 OS filesystem heavily trust하고 AWS RDS 스토리지 자체가 추상화, Aurora는 AZ 분산샤딩 지원.
Block구조	[Block Header] [ITL] [Table Directory] [Row Directory] [Free Space] [Row Data]	[PageHeader] [ItemId array(line pointer)] [Free Space] [Tuple Data]	오라클 Block = PG Page or Block
Block철학	"같은 row를 블록 안에서 가능한 오래 유지"	"단순히 새 row version append로 계속 생성 - append only"	Heavy append only시 단순 로직인 PG가 더 유리함
Block여유공간	PCTFREE 20 블록의 20%는 미래 UPDATE 용 공간으로 row chain 방지	fillfactor = 80 페이지를 70%만 채움, hot update 방지	INSERT만 발생하는 로그/이력성 append only 테이블이면 여유공간 0% 권장 (oracle, pg 동일)
CLI Client	SQL*Plus 오래된 과거형 UNIX 스타일	psql 최신 간단하고 강력한 LINUX	PG는 \gexec로 select결과 바로 실행기능 및 \command 강력한

		스타일	스타일
데이터 이관	동기종간 impdp / expdp 이기종간 SQL*Loader / Plus STDIO 파이프 처리 불편함	동/이기종간 psql \copy 하나로 가능 STDIO 파이프 처리 심플함	오라클의 경우 캐릭터셋 제한이 많음 us7ascii, ksc5601, mswin949, al16utf8, al32utf8 등등
대량**INSERT**	undo/redo, buffer pin, ITL, freelist/ASSM 관여로 RAC에서 성능급감 *index leaf contention* *buffer busy waits* *gc current block busy*	heap page append, sequential WAL 기록만 이뤄지고 애초에 MVCC모델이 append only 에 최적화 되어서 성능이 상당히 빠르지만 INSERT 이후에 vacuum 부담 발생	Oracle SQL*Loader direct path 나 /*+ APPEND */ + NOLOGGING 은 상당히 빠름 PG는 \copy 나 INSERT 모두 별도 설정 없이 상당히 빠름
ORA-01555: snapshot too old	Oracle초창기 버전 부터 최신 버전까지 유명한 고질병 "읽어야 할 과거 버전이 undo에서 지워짐"	동일한 heap내에서 여러 버전을 관리하여 undo segment가 없는 구조로 오류 발생할 가능성 없음	*1. 2*시간짜리 대형 *SELECT* 수행 *2.* 그동안 다른 세션 *UPDATE* 폭증 *3. undo* 공간 재사용 *overwrite* *4. SELECT*가 필요한 과거 *row version* 사라짐 *5.* 오류발생, *2*시간 전으로 돌아감
재구성 **(Reorganization)**	DBA가 수동으로 주기적인 재구성 필요 *expdp* 이후 *impdp* *tablespace* 이관 *(MOVE TABLESPACE)* *SHRINK* “overwrite구조의 단편화정리”	auto vacuum이 dead tuple을 수시로 정리하지만 과도한 bloat발생 및 단편화 발생 시에만 DBA가 수동으로 재구성 필요 *vacuum full* *pg_repack* “MVCC dead tuple 청소”	재구성 이후 OS공간 확보 방법 Oracle shrink 이후 datafile resize *datafile*끝에 남으면 *resize*실패, *ts*의 첫번째 *datafile*손상시 *ts drop*해야됨 PG *vacuum full* 또는 *pg_repack rewrite* *table oid* 변경됨, 삭제된 파일인데 프로세스 *fd* 잡고 있음 공간확보 안됨
Vector타입	Oracle 23ai버전 부터 지원됨 *CREATE TABLE docs (* *id NUMBER,* *content CLOB,* *embedding VECTOR(1536, FLOAT32)* *);*	pg_vector extension 설치 후 무료사용 *create extension pg_vector;* *CREATE TABLE docs (* *id NUMBER,* *content TEXT,* *embedding VECTOR(4)* *);*	사실상 pgvector가 오라클보다 표준처럼 사용됨, 특히 python 호환율이 높아 RAG처리 강력함. *ex> python code* *embedding = openai.embeddings.create(...)*
GIS	Oracle Spatial 비용비쌈. 대형 엔터프라이즈 급에서 검증됨 *CREATE TABLE places (* *id NUMBER GENERATED BY DEFAULT AS IDENTITY,* *name VARCHAR2(100),* *geom MDSYS.SDO_GEOMETRY* *);* *select SDO_RELATE(* *a.geom,* *b.geom,* *'mask=ANYINTERACT'* *) = 'TRUE'...*	postgis extension 설치 후 무료사용 *create extension pg_vector;* *CREATE TABLE places (* *id SERIAL,* *name TEXT,* *geom geometry(Point, 4326)* *);* *select ST_Intersects(a.geom, b.geom)...*	사실상 postgis가 GIS 표준, 강력한 geography 함수들을 오라클이 따라감. *ST_Intersects* *ST_Contains* *ST_Buffer* *ST_Union* *ST_DWithin* *ST_Simplify*

유사도검색(**cosine similarity)**	Oracle 23ai버전 부터 지원됨 *SELECT ** *FROM docs* *ORDER BY VECTOR_DISTANCE(* *embedding,* *:query_vector,* *COSINE* *)* *FETCH FIRST 10 ROWS ONLY;*	pg_vector extension 설치 후 무료사용 *SELECT* *id,* *content,* *embedding <=> '[0.1,0.2,0.3,0.39]' AS distance* *FROM documents* *ORDER BY embedding <=> '[0.1,0.2,0.3,0.39]'* *LIMIT 10;*	pgvector가 좋은 이유 - 기존 SQL 그대로 사용 - JOIN 가능 - transaction 가능 - metadata filtering 가능